

My Report

Class methods:

These are methods written on the class side. They belong to the class, not the objects it creates.

Example code :

```
Counter class >> withValue: n
  ^ self new value: n; yourself.
```

If I forget the ^, it just gives back the class, not a new Counter. Once I add ^, it works fine.

Super:

super means “look in my parent class,” but it still sends the message to the same object.

Example code :

```
B >> sayHello
  ^ super sayHello
```

It starts looking in A (the parent), but it's still the same object talking.

Stone-Paper-Scissors:

I learned that instead of checking conditions like “if stone vs paper,” each object can decide what to do by sending another message to the other object. For example, Stone sends playAgainstStone to Paper, and Paper answers who wins. This is called double dispatch because two messages are sent :

- one to choose the right vs: method

- one to choose the correct playAgainst... method

It makes the code cleaner and easier to extend later, like adding Lizard or Spock, without changing old code.

GameView:

I saw how double dispatch helps the game draw different types of blocks on the screen without having a big list of if checks. Each block knows how to tell the view what to draw.

Conclusion:

Class methods are just messages sent to the class itself, and super doesn't change the object, it only tells Pharo to keep searching for the method in the parent class. Trying out small code examples, and both the stone-paper-scissors and gameview exercises, really helped me see how messages move between classes and how Pharo decides which method to run.